

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	M. Mohamed Mohaiyadeen	Reg. No.:	192424134
Section / Batch:	3 rd Year AI - DS	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1 — Unsmoothed N-gram Language Model

```
import re
from collections import Counter

corpus = """
The student is reading a book.
The student is writing an assignment.
The student is learning Python.
The student likes machine learning.
The teacher is teaching the student.
The teacher is reading a book.
The student reads the book.
The student works on a project.
"""

def tokenize(text):
    s = re.split(r'[!/?]+', text.lower())
    return ['<s>' + re.findall(r'\b[a-z]+\b', x) + ['</s>']
            for x in s if x.strip()]

sentences = tokenize(corpus)

uni, bi, tri = Counter(), Counter(), Counter()

for s in sentences:
    uni.update(s)
    bi.update(zip(s, s[1:]))
    tri.update(zip(s, s[1:], s[2:]))

total = sum(uni.values())

def prob(ngram):
    if len(ngram) == 1:
        return uni[ngram[0]] / total
    if len(ngram) == 2:
        return bi[ngram] / uni[ngram[0]] if uni[ngram[0]] else 0
    return tri[ngram] / bi[ngram[:2]] if bi[ngram[:2]] else 0

def predict(text, n):
    w = re.findall(r'\b[a-z]+\b', text.lower())
    candidates = []

    for word in uni:
        if n == 1:
            p = prob((word,))
        elif n == 2 and w:
            p = prob((w[-1], word))
        elif n == 3 and len(w) >= 2:
            p = prob((w[-2], w[-1], word))
        else:
            p = 0
        if p > 0:
            candidates.append((word, p))

    return sorted(candidates, key=lambda x: x[1],
                  reverse=True)[:5]

while True:
    print("\n1.Unigram 2.Bigram 3.Trigram 4.Predict
5.Unseen 6.Test 7.Exit")
    ch = input("Choice: ")

    if ch in "123":
        n = int(ch)
        counts = [uni, bi, tri][n - 1]
        print("\nCounts and probabilities:")
        for g, c in counts.items():
            print(g, ":", c, "->", round(prob(g if isinstance(g,
tuple) else (g,)), 4))

    elif ch == "4":
        n = int(input("N (1/2/3): "))
        text = input("Sentence: ")
        print("Top-5:", predict(text, n))

    elif ch == "5":
        print("P(student beautiful) =", prob(("student",
"beautiful")))
        print("P(student is beautiful) =", prob(("student", "is",
"beautiful")))
        print("Unseen N-grams have probability 0.")

    elif ch == "6":
        for text in ["the student is", "the teacher is", "the student
likes"]:
            print(text, "->", predict(text, 2))

    elif ch == "7":
        break
```

Output:

```
=====
      N-GRAM LANGUAGE MODEL
=====
1. Unigram
2. Bigram
3. Trigram
4. Predict next word
5. Test unseen N-gram
6. Evaluate model
7. Exit
Enter your choice: 6

--- PREDICTION EVALUATION ---

Input: the student is
Predicted next words:
reading -> 0.1667
writing -> 0.1667
learning -> 0.1667
studying -> 0.1667
explaining -> 0.1667

Input: the teacher is
Predicted next words:
reading -> 0.1667
writing -> 0.1667
learning -> 0.1667
studying -> 0.1667
explaining -> 0.1667

Input: the student likes
Predicted next words:
machine -> 0.5
computer -> 0.5
```

```
=====
      N-GRAM LANGUAGE MODEL
=====
1. Unigram
2. Bigram
3. Trigram
4. Predict next word
5. Test unseen N-gram
6. Evaluate model
7. Exit
Enter your choice: 4
Enter N (1, 2 or 3): 2
Enter incomplete sentence: The student is

Top-5 predictions:
reading -> Probability: 0.1667
writing -> Probability: 0.1667
learning -> Probability: 0.1667
studying -> Probability: 0.1667
explaining -> Probability: 0.1667
```

Task 2: Backoff and Deleted Interpolation

```
import re
from collections import Counter

corpus = """
the student is reading a book
the student is writing an assignment
the student is learning python
the student likes machine learning
the teacher is teaching the student
the teacher is reading a book
the student reads the book
the student works on a project
"""

sentences = [['<s>'] + re.findall(r'\b[a-z]+\b',
s.lower()) + ['</s>']
              for s in corpus.strip().split('\n')]

uni, bi, tri = Counter(), Counter(), Counter()

for s in sentences:
    uni.update(s)
    bi.update(zip(s, s[1:]))
    tri.update(zip(s, s[1:], s[2:]))

total = sum(uni.values())

def p1(w):
    return uni[w] / total

def p2(a, b):
    return bi[a, b] / uni[a] if uni[a] else 0

def p3(a, b, c):
    return tri[a, b, c] / bi[a, b] if bi[a, b] else 0

# Backoff: trigram -> bigram -> unigram
def backoff(a, b, c):
    if p3(a, b, c): return p3(a, b, c), "Trigram"
    if p2(b, c): return p2(b, c), "Bigram"
    return p1(c), "Unigram"

# Deleted interpolation
l1, l2, l3 = 0.2, 0.3, 0.5
```

```
def interpolate(a, b, c):
    return l1*p1(c) + l2*p2(b, c) + l3*p3(a, b, c)

def predict(text, model):
    w = re.findall(r'\b[a-z]+\b', text.lower())
    if len(w) < 2:
        return None

    result = []
    for x in uni:
        if model == "unsmoothed":
            p = p3(w[-2], w[-1], x)
        elif model == "backoff":
            p, _ = backoff(w[-2], w[-1], x)
        else:
            p = interpolate(w[-2], w[-1], x)
        result.append((x, p))

    return max(result, key=lambda x: x[1])

while True:
    print("\n1.Unsmoothed 2.Backoff 3.Interpolation
4.Compare 5.Exit")
    ch = input("Choice: ")

    if ch == "5":
        break

    text = input("Enter sentence: ")

    if ch in "123":
        model = ["unsmoothed", "backoff",
"interpolation"][int(ch)-1]
        ans = predict(text, model)
        print("Prediction:", ans)

    if model == "backoff":
        w = re.findall(r'\b[a-z]+\b', text.lower())
        print("Used:", backoff(w[-2], w[-1], ans[0])[1])

    elif ch == "4":
        for model in ["unsmoothed", "backoff",
"interpolation"]:
            print(model, ":", predict(text, model))
```

Output:

1.Unsmoothed 2.Backoff 3.Interpolation 4.Compare 5.Exit

Choice: 2

Enter sentence: I am a student from

Prediction: ('the', 0.16393442622950818)

Used: Unigram

1.Unsmoothed 2.Backoff 3.Interpolation 4.Compare 5.Exit

Choice: 3

Enter sentence: Pizza is my favourite

Prediction: ('the', 0.03278688524590164)

1.Unsmoothed 2.Backoff 3.Interpolation 4.Compare 5.Exit

Choice: 4

Enter sentence: Hi how are you

unsmoothed : ('<s>', 0)

backoff : ('the', 0.16393442622950818)

interpolation : ('the', 0.03278688524590164)

Task 3: N – gram Entropy

```
import re
import math
from collections import Counter

train = [
    "the student is reading a book",
    "the student is writing an assignment",
    "the student is learning python",
    "the student likes machine learning",
    "the teacher is teaching the student"
]

test = [
    "the student is reading a book",
    "the student likes machine learning",
    "the teacher is teaching the student",
    "the student plays football"
]

train = ['<s>'] + re.findall(r'\b[a-z]+\b', s) + ['</s>'] for
s in train]
test = ['<s>'] + re.findall(r'\b[a-z]+\b', s) + ['</s>'] for
s in test]

uni, bi, tri = Counter(), Counter(), Counter()

for s in train:
    uni.update(s)
    bi.update(zip(s, s[1:]))
    tri.update(zip(s, s[1:], s[2:]))

V = len(uni)
N = sum(uni.values())

def probability(s, i, n, smooth=False):
    w = s[i]

    if n == 1:
        return (uni[w] + smooth) / (N + smooth*V)
    if n == 2:
```

```
        c = bi[s[i-1], w] + smooth
        d = uni[s[i-1]] + smooth*V
    else:
        c = tri[s[i-2], s[i-1], w] + smooth
        d = bi[s[i-2], s[i-1]] + smooth*V

    return c / d if d else 0

def entropy(s, n, smooth=False):
    values = []
    for i in range(1, len(s)):
        p = probability(s, i, n, smooth)
        if p == 0:
            return float("inf")
        values.append(-math.log2(p))
    return sum(values) / len(values)

for n in [1, 2, 3]:
    print("\nN =", n)

    for s in test:
        print(
            " ".join(s),
            "->",
            entropy(s, n)
        )

    print("\n--- Smoothed Entropy ---")

    for n in [1, 2, 3]:
        values = [entropy(s, n, True) for s in test]
        print("N =", n, "Average =",
              round(sum(values)/len(values), 4))

    print("\nInterpretation:")
    print("Low entropy = more predictable words.")
    print("High entropy = less predictable words.")
    print("Smoothing avoids zero probabilities for
    unseen N-grams.")
```

Output:

```
N = 1
<s> the student is reading a book </s> -> 3.9295248433727457
<s> the student likes machine learning </s> -> 3.876457731694272
<s> the teacher is teaching the student </s> -> 3.560244486126866
<s> the student plays football </s> -> inf

N = 2
<s> the student is reading a book </s> -> 0.42857142857142855
<s> the student likes machine learning </s> -> 0.5974937501201927
<s> the teacher is teaching the student </s> -> 1.024275000206044
<s> the student plays football </s> -> inf

N = 3
<s> the student is reading a book </s> -> inf
<s> the student likes machine learning </s> -> inf
<s> the teacher is teaching the student </s> -> inf
<s> the student plays football </s> -> inf

--- Smoothed Entropy ---
N = 1 Average = 3.8862
N = 2 Average = 2.9428
N = 3 Average = 3.3571

Interpretation:
Low entropy = more predictable words.
High entropy = less predictable words.
Smoothing avoids zero probabilities for unseen N-grams.
```

Task 4 – POS Tagging

```
import re
from collections import Counter, defaultdict

train = [
    [("the", "DT"), ("student", "NN"), ("reads", "VBZ"), ("a", "DT"), ("book", "NN")],
    [("the", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("the", "DT"), ("student", "NN")],
    [("she", "PRP"), ("reads", "VBZ"), ("the", "DT"), ("book", "NN")],
    [("he", "PRP"), ("likes", "VBZ"), ("the", "DT"), ("book", "NN")],
    [("students", "NNS"), ("are", "VBP"), ("learning", "VBG"), ("python", "NN")],
    [("the", "DT"), ("student", "NN"), ("is", "VBZ"), ("happy", "JJ")],
    [("the", "DT"), ("student", "NN"), ("works", "VBZ"), ("hard", "RB")]
]

lexicon = dict(
    x for s in train for x in s
)

def words(text):
    return re.findall(r'\b[a-z]+\b', text.lower())

def rule_tagger(text):
    ans = []
    for w in words(text):
        if w in lexicon:
            tag = lexicon[w]
        elif w.endswith("ly"):
            tag = "RB"
        elif w.endswith("ing"):
            tag = "VBG"
        elif w.endswith("ed"):
            tag = "VBD"
        elif w.endswith("s"):
            tag = "NNS"
        else:
            tag = "NN"
        ans.append((w, tag))

    return ans

emit = defaultdict(Counter)
trans = defaultdict(Counter)
tags = set()

for s in train:
    prev = "<S>"
    for w, t in s:
        emit[t][w] += 1
        trans[prev][t] += 1
        tags.add(t)
        prev = t

def stochastic(text):
    ws = words(text)
    score, path = {}, {}

    for t in tags:
        score[t] = trans["<S>"][t] * emit[t][ws[0]]
        path[t] = [t]

    for w in ws[1:]:
```

```
        new, newpath = {}, {}

        for t in tags:
            best = max(
                ((score[p] * (trans[p][t]+1) *
                  (emit[t][w]+1)), p) for p in tags
            )
            new[t] = best[0]
            newpath[t] = path[best[1]] + [t]

        score, path = new, newpath

    best = max(score, key=score.get)
    return list(zip(ws, path[best]))

def transformation(text):
    result = rule_tagger(text)

    for i, (w, t) in enumerate(result):
        prev = result[i-1][1] if i else ""

        # Pronoun + noun -> verb
        if prev == "PRP" and t in ("NN", "NNS"):
            result[i] = (w, "VB")

        # Auxiliary + noun -> verb
        elif prev in ("VBZ", "VBP") and t in ("NN", "NNS"):
            result[i] = (w, "VB")

    return result

def show(title, result):
    print(title, ":", " ".join(w+"/"+t for w,t in result))

while True:
    print("\n1.Rule Based 2.Stochastic\n3.Transformation 4.Compare 5.Exit")
    ch = input("Choice: ")

    if ch == "5":
        break

    text = input("Enter sentence: ")

    if ch == "1":
        show("Rule Based", rule_tagger(text))

    elif ch == "2":
        show("Stochastic", stochastic(text))

    elif ch == "3":
        show("Transformation", transformation(text))

    elif ch == "4":
        show("Rule Based", rule_tagger(text))
        show("Stochastic", stochastic(text))
        show("Transformation", transformation(text))
```


Output:

1.Rule Based 2.Stochastic 3.Transformation 4.Compare 5.Exit
Choice: 1

Enter sentence: I am sick

Rule Based : i/NN am/NN sick/NN

1.Rule Based 2.Stochastic 3.Transformation 4.Compare 5.Exit
Choice: 2

Enter sentence: This is a beautiful peacock

Stochastic : this/VBZ is/VBZ a/VBZ beautiful/VBZ peacock/VBP

1.Rule Based 2.Stochastic 3.Transformation 4.Compare 5.Exit
Choice: 3

Enter sentence: I like to run daily

Transformation : i/NN like/NN to/NN run/NN daily/RB

1.Rule Based 2.Stochastic 3.Transformation 4.Compare 5.Exit
Choice: 4

Enter sentence: Nice to meet you MR Trump

Rule Based : nice/NN to/NN meet/NN you/NN mr/NN trump/NN

Stochastic : nice/VBZ to/VBZ meet/VBZ you/VBZ mr/VBZ trump/VBP

Transformation : nice/NN to/NN meet/NN you/NN mr/NN trump/NN

1.Rule Based 2.Stochastic 3.Transformation 4.Compare 5.Exit
Choice: 5

PS C:\COURSE\DSA0302\CO3_AT1> █

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____